

TP — TypeScript & React (M2)

Objectif

Vous allez construire progressivement une petite application React en TypeScript permettant de gérer une bibliothèque de livres. Vous pouvez utiliser ViteJs avec la configuration React + Typescript

Le TP est structuré en plusieurs parties. Chaque partie introduit une notion de typage.

Consignes :

- Essayez d'écrire le code **avant d'ajouter les types**
- Puis ajoutez les types et corrigez les erreurs TypeScript
- N'utilisez pas `any`

Contexte du mini-projet

Vous travaillez pour une petite bibliothèque qui souhaite moderniser un ancien système de gestion encore basé sur des outils peu adaptés. L'objectif est de concevoir une application web simple permettant de gérer une collection de livres et de suivre leur disponibilité.

Au cours de ce TP, vous allez construire progressivement cette application en partant d'une structure de données basique, puis en ajoutant des composants React, de la gestion d'état et des fonctionnalités interactives. Chaque étape correspond à une évolution réaliste du projet, comme cela se ferait dans un contexte professionnel.

L'accent est mis sur la qualité du typage avec TypeScript, la clarté du code et la structuration globale de l'application. L'interface utilisateur reste libre, mais le code devra être compréhensible et maintenable, comme s'il devait être repris par un autre développeur.

Vous êtes libres sur l'interface (UI), tant que les fonctionnalités demandées sont présentes.

PARTIE 1 — Interfaces & dictionnaires

Exercice 1

Créer une interface `Book` avec les propriétés suivantes :

- `id`
- `title`
- `author`

Créer ensuite une structure de données permettant de stocker plusieurs livres sous forme de dictionnaire.

Exercice 2

Ajouter une propriété permettant d'indiquer si un livre est disponible.

Créer une fonction permettant de récupérer uniquement les livres disponibles.

PARTIE 2 — Typage des composants React

Exercice 3

Créer un composant `BookItem` qui affiche les informations d'un livre.

Exercice 4

Créer un composant `BookList` qui :

- prend une liste de livres en entrée
- affiche tous les livres

PARTIE 3 — `useState`

Exercice 5

Dans un composant principal :

- créer un state pour stocker des livres
- ajouter un bouton permettant d'ajouter un livre

Exercice 6

Créer un formulaire permettant de saisir :

- un titre
- un auteur

Lors de la soumission :

- ajouter le livre au state

PARTIE 4 — Mini projet

Exercice 7

Modifier votre structure de données pour utiliser un dictionnaire au lieu d'un tableau.

Adapter le code existant.

Exercice 8

Ajouter les fonctionnalités suivantes :

Filtre

Permettre de filtrer les livres par titre.

Sélection

Permettre de sélectionner un livre et afficher ses détails.

Statut

Ajouter un statut au livre :

- disponible
- emprunté

Ajouter un bouton permettant de changer ce statut.

Bonus

Créer un composant réutilisable permettant d'afficher une liste d'éléments quelconque.

Attendus

À la fin du TP, vous devez avoir :

- une application fonctionnelle
- du code typé proprement
- un répertoire Github contenant votre projet, si possible en respectant les bonnes pratiques de versionning.
- aucune utilisation de `any`

Grille d'évaluation



Grille d'évaluation — TP TypeScript & React (/20)

Critère	Détail	Points
React	Fonctionnalités (affichage, ajout, filtre, sélection)	/3
	Découpage en composants & props	/2
	Utilisation des hooks (useState, logique)	/1
Sous-total React		/6
TypeScript	Typage des données (interfaces, dictionnaire)	/3
	Typage des composants & props	/2
	Typage du state	/2
	Qualité globale (cohérence, pas de any)	/1
Sous-total TypeScript		/8
GitHub	Commits + messages + repo fonctionnel	/1
Structure	Organisation des fichiers & clarté	/0,5
Style	Lisibilité, nommage, propreté	/0,5
TOTAL		/20

Pénalités possibles

- Utilisation de **any** : -1 à -3
- Application non fonctionnelle : jusqu'à -5
- Code incohérent / non compris : pénalité globale